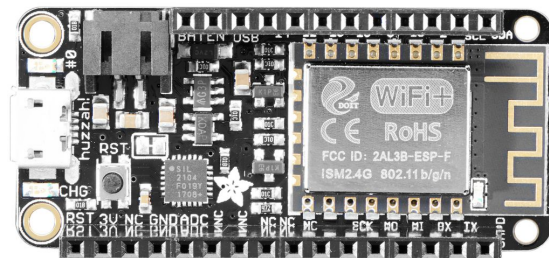
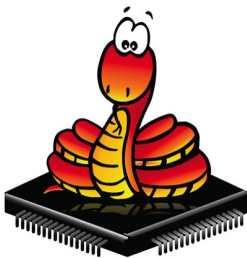


Internet of Things - Intelligent and Connected Systems

EECS E4764
Columbia University

Prof. Xiaofan Jiang

Fall 2022
Lab 5 - Embedded Servers



Introduction

As the name implies, a server is a component or device that provides a service or function to a client device. Any web page you can access through the Internet browser and applications, like games, and video services are hosted on servers for users and clients to request and interact with. All a basic server does is take in requests from a client and sends back a response.

In this lab, our objective is to add voice commands to our smartwatch. We will start by making a simple smartphone app that enables voice recognition. Then, we will make our wearable application (more specifically the ESP8266/Huzzah) into a server that takes requests from the smartphone app through HTTP. More specifically, the smartphone app will send commands or requests to the Huzzah server, and the Huzzah will execute these commands to perform actions.

Part 1: Smartphone Application

Task

Create a smartphone application that interfaces with voice recognition API. You can either use smartphone's built-in voice-to-text in the SDK or some online API. The application should allow users to speak a voice command and translate it into text. Later we will send these commands to the ESP8266 server via HTTP.

If you have worked with app developments before, feel free to use the platform you are familiar with. Otherwise, making an Android App with Android Studio is a good starting point. There are a lot of tutorial videos/articles online on "Android Studio voice to text". (like https://www.youtube.com/watch?v=pjK36SmQ_9A)

Checkpoint 1

Make a simple application that enables voice recognition.
Display the text on the screen after speaking to the microphone.

(You can skip the rest of this part if you are not using Android Studio)

To download Android Studio, go to

<https://developer.android.com/studio/index.html>

and download the version appropriate for your operating system.

General Tips for Running and Debugging Android applications:

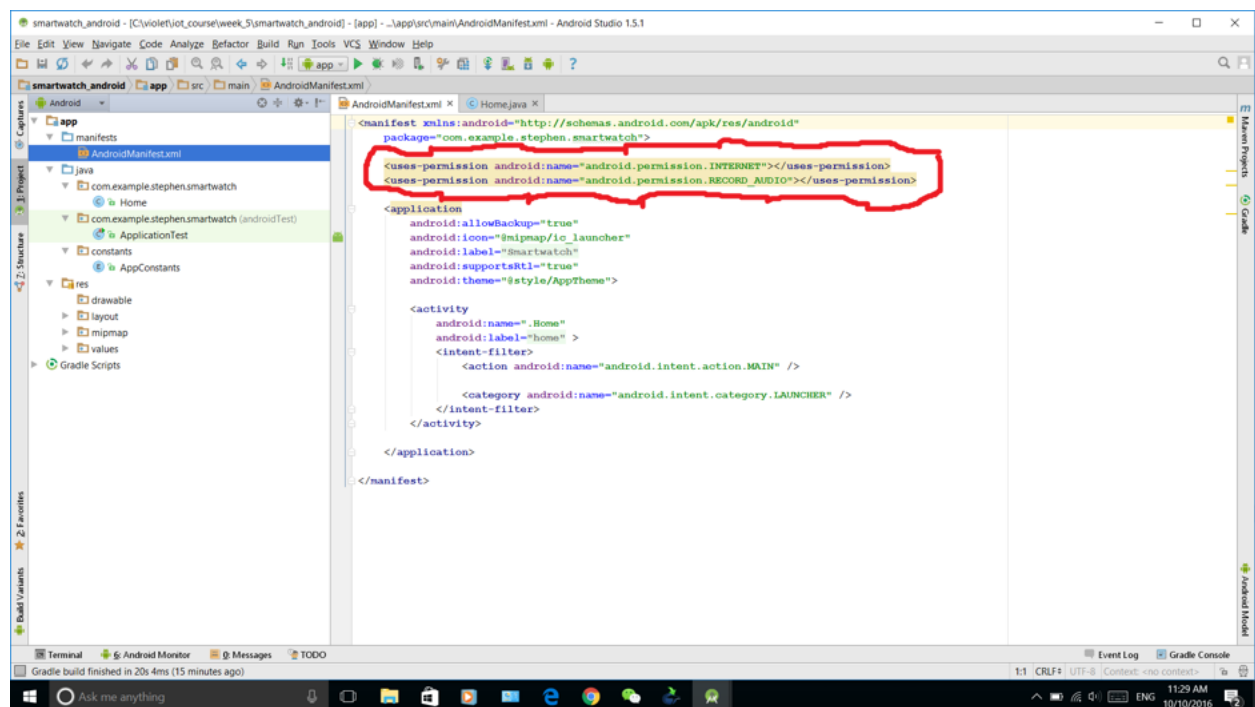
Developer Options:

In order to download applications to your device from Android Studio, you must enable developer options and USB Debugging on your device. To do this, go to **Settings->About Phone** and tap **Build number** about 7 times until you get a message saying that "developer options" has been enabled. Once this is enabled, you should see a new menu item called **Developer options** appear in your Settings screen. Go into this menu and enable **USB debugging**.

Android Manifest

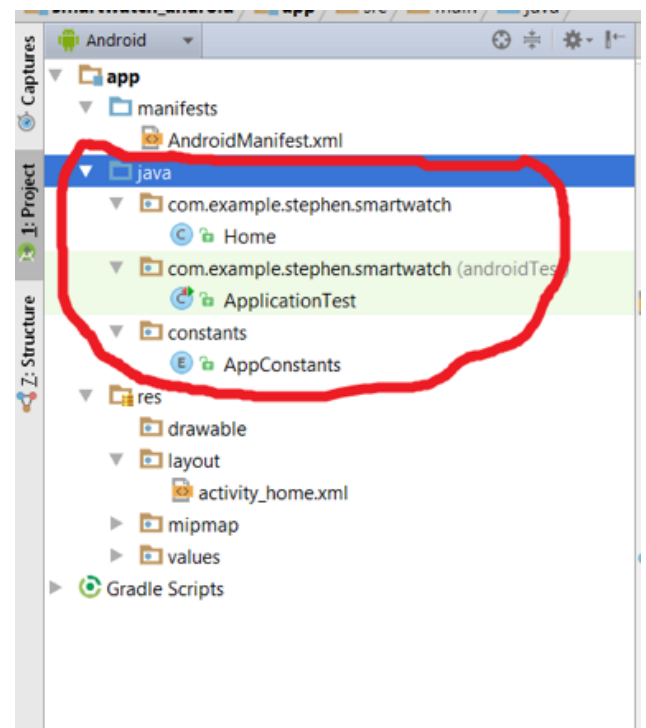
The **AndroidManifest.xml** file contains information that specifies how the application is set up (permissions the application requires, what activities are within the application, which activity is the main activity, etc.). For this lab, you must add the

Internet permission and the speech recording permission because the application will record your voice and communicate to Google Speech API and the HUZAH through the Internet. The image provided below details what needs to be added.



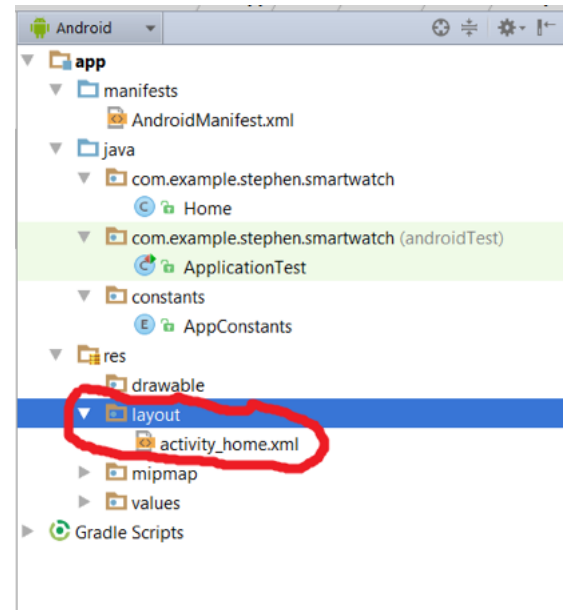
Activities and Source Code

Android applications run and allow users to interact with them through activities. You can specify what your application does through activities, specified in **app->java**. Essentially, the files in this folder contain the “source code” that specifies how your application functions.



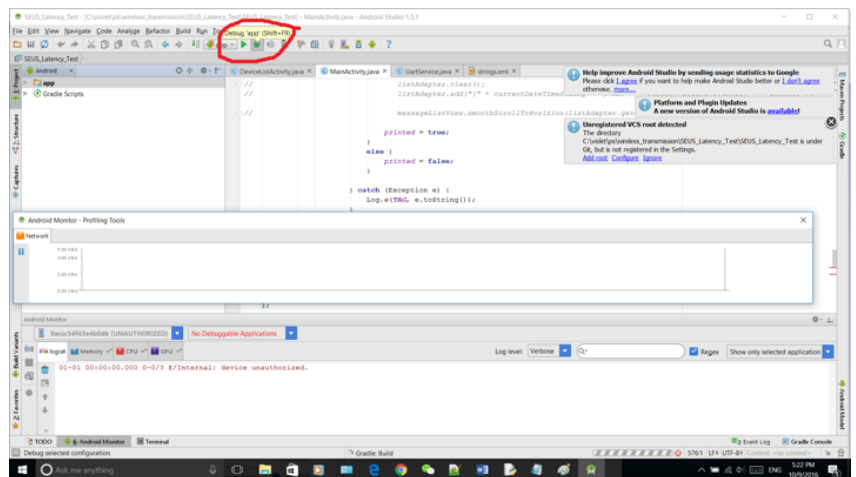
Activity Layouts

The java source files specify how your application functions, but any smartphone application also has a user interface that users can interact with. User interfaces can be created in .xml files located in **res->layout**. These screens must be linked to an activity specified in **app->java**.

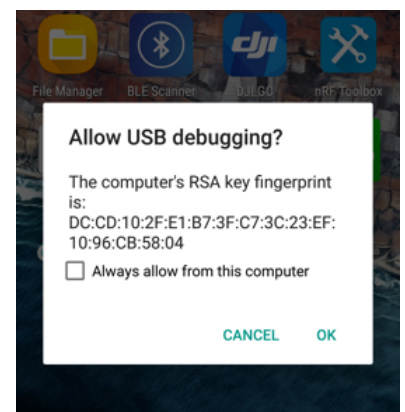


Downloading the Application to your Device

Once you have enabled USB debugging on your phone and you have created your application, click on the phone icon in the toolbar to launch the application on a virtual device or connect your phone to the computer. Click on the debug button, as shown below:



Once you click on the debug button, Android Studio will launch something called the “Android Debug Bridge” or ADB, which essentially allows you to transfer your application from Android Studio to your device. When the ADB launches, your phone/emulator will receive a notification, like the one shown below. You must accept in order for you to be able to load your application onto your phone.



Resources

<https://developer.android.com/develop/index.html>

<http://www.instructables.com/id/How-To-Create-An-Android-App-With-Android-Studio/>

<https://developer.android.com/guide/topics/manifest/manifest-intro.html>

<https://developer.android.com/reference/android/app/Activity.html>

<https://developer.android.com/guide/topics/ui/declaring-layout.html>

<https://developer.android.com/guide/topics/ui/controls/button.html>

<http://android4beginners.com/2013/06/lesson-1-how-to-display-and-format-a-text-in-android-apps/>

(For next part)

<https://developer.android.com/training/basics/network-ops/connecting.html>

<https://developer.android.com/reference/org/json/JSONObject.html>

https://www.tutorialspoint.com/android/android_json_parser.htm

Part 2: Embedded Server on Huzzah

Task 1

On the ESP8266, implement a web server using sockets, that will allow it to receive and process HTTP requests.

Task 2

Interface the smartphone app with the smartwatch embedded server. Configure the display to turn off and turn on, display the time, and display any other message on the OLED whenever it receives the corresponding command from the smartphone. (See specific requirements below)

In order to access a server via HTTP on the Internet, the server must have a global IP address, which the ESP8266 does not have by default; it only has a local IP. To get around this, use ngrok to create a tunnel connection to the ESP8266. The ngrok software is available here: <https://ngrok.com/>. More information on how to use ngrok can be found here: <https://ngrok.com/docs>.

Specifications:

1. Your system should be able to handle the following commands:
 - “display on/off”: turn on or off the OLED display
 - “display time”: behave like a regular digital watch (the time on the OLED should change corresponding to the actual time that has passed in the real-world).
 - Other messages: display the message on the OLED display
2. In a system like this, there should be 2-way communication between devices; when a command is sent to the Huzzah, the Huzzah should send back a response to the smartphone. The response should be displayed on the user interface of the application, and it should provide information about whether or not the command was successfully received and interpreted.

Checkpoint 2

Use the voice commands to turn off/on the display, display the time, and display any other spoken message on the OLED screen.

Lab 5 Checkpoints:

1. Make a simple application that enables voice recognition. Display the text on the screen after speaking to the microphone.
2. Use the voice commands to turn off/on the display, display the time, and display any other spoken message on the OLED screen.

Submission:

1. Zip the folder containing the mobile App
2. Submit the zip file along with code for final checkpoint onto courseworks → Assignments → Lab5

The submitted files should be named like

`labx_{member1_uni}_{member2_uni}_{member3_uni}_{description}.py/zip`

There should be 2 files for this lab.

e.g.

- Lab5_kh3119_sm4928_rl3154_app.zip
- Lab5_kh3119_sm4928_rl3154_server.py